

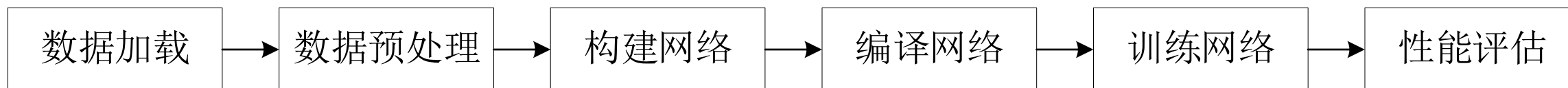


数据加载与预处理



PyTorch深度学习通用流程

- 深度学习通常包含数据加载与预处理、构建网络、训练网络和性能评估几个主要的步骤。
- PyTorch为每个步骤提供了一些相应的函数，使得可以方便快速地进行深度学习。
- 以猫狗分类的例子介绍PyTorch深度学习的通用流程。
- 深度学习的通用流程如图所示，分为以下6个步骤。



深度学习的通用流程分为以下6个步骤。

- **数据加载**，加载用于训练深度神经网络的数据，使得深度神经网络能够学习到数据中潜在的特征，可以指定从**某些特定路径加载数据**。
- **数据预处理**，对加载的数据进行预处理，使之**符合网络的输入要求**，如标签格式转换、样本变换等。数据形式的不统一将对模型效果造成较大的影响。
- **构建网络**，根据特定的任务使用不同的**网络层搭建网络**，若网络太简单则无法学习到足够丰富的特征，若网络太复杂则容易过拟合。

- **编译网络**，设置网络训练过程中使用的**优化器和损失函数**，优化器和损失函数的选择会影响到网络的训练时长、性能等。
- **训练网络**，通过不断的**迭代和批训练**的方法，调整模型中各网络层的参数，减小模型的损失，使得模型的预测值逼近真实值。
- **性能评估**，计算网络训练过程中**损失和分类精度**等与模型对应的评价指标，根据评价指标的变化调整模型从而取得更好的效果。

目录



数据加载与预处理

- 数据的形式多种多样，因此读取数据的方法也是多种多样。
- 在一个图像分类的任务中，图像所在的文件夹即为图像的类别标签，则需要把图片和类别标签都读入计算机中，同时还需对图像数据进行张量变换和归一化等预处理。
- PyTorch框架中提供了一些常用的数据读取和预处理的方法。

数据加载

- 在PyTorch框架中，`torchtext.utils`包中的类可以用于数据加载。
- 常见加载的数据的方式包括从指定的网址（`url`）下载数据、加载本地的表格数据和直接读取压缩文件中的数据。

1. 从指定的网址下载文件

- `download_from_url`类可以从对应网址下载文件并返回所下载文件的存储路径。
- `download_from_url`类的语法格式如下。

```
torchtext.utils.download_from_url(url, path=None, root='.data',  
overwrite=False, hash_value=None, hash_type='sha256')
```

- `download_from_url`类的常用参数及其说明如表所示。

1. 从指定的网址下载文件

参数名称	说明
url	接收str，表示url文件的网络路径，无默认值
root	接收str，表示用于存放下载数据文件的文件夹路径，无默认值
overwrite	接收bool，表示是否覆盖当前文件，默认为False

1. 从指定的网址下载文件

```
import torchtext
```

```
url =
```

```
    'https://mirrors.cloud.tencent.com/pypi/packages/51/59/d190ffe6fac  
2f5c7a301bd9ffd5f69b9a1925e08dbfec6ee1e8b816fedb/torchtext-  
0.9.0-cp38-cp38-win_amd64.whl'
```

```
torchtext.utils.download_from_url(url, '../data/torchtext.whl')
```

2. csv文件读取器

- `unicode_csv_reader`类用于读取csv数据文件，`unicode_csv_reader`类的语法格式如下。
- 其中的参数“`unicode_csv_data`”指的是csv数据文件。

`torchtext.utils.unicode_csv_reader(unicode_csv_data, **kwargs)`

2. csv文件读取器

```
from torchtext.utils import unicode_csv_reader  
import io
```

```
data_path = '../data/city_economy.csv'  
with io.open(data_path, encoding='utf8') as f:  
    reader = unicode_csv_reader(f)
```

3. 读取压缩文件数据

- `extract_archive`类用于读取压缩文件中的数据。
- `extract_archive`类的语法格式如下。

`torchtext.utils.extract_archive(from_path, to_path=None, overwrite=False)`

- `extract_archive`类的常用参数及其说明如表所示。

参数名称	说明
<code>from_path</code>	接收str，表示数据文件的路径，无默认值
<code>to_path</code>	接收str，表示提取文件的根路径，无默认值
<code>overwrite</code>	接收bool，表示是否覆盖当前文件，无默认值

2. 读取压缩文件数据

```
from_path = '../data/test_data.zip'  
to_path = '../data/'  
torchtext.utils.download_from_url(url, from_path)  
torchtext.utils.extract_archive(from_path, to_path)
```

目录



数据预处理

- 在深度学习的时候，可以事先单独将图片进行清晰度、画质和切割等处理然后存起来以扩充样本，但是这样做效率比较低，而且不是实时的。
- 接下来介绍如何用PyTorch对数据进行处理。
- 数据预处理分为图像数据预处理和文本数据预处理。

图像数据预处理

在PyTorch框架中，处理图像与视频的torchvision库中常用的包及其说明如表所示。

包	说明
torchvision.datasets	提供数据集下载和加载功能，包含若干个常用数据集
torchvision.io	提供执行IO操作的功能，主要用于读取和写入视频及图像
torchvision.models	包含用于解决不同任务的网络结构，并提供已预训练过的模型，包括图像分类、像素语义分割、对象检测、实例分割、人物关键点检测和视频分类
torchvision.ops	主要实现特定用于计算机视觉的运算符
torchvision.transforms	包含多种常见的图像预处理操作，如随机切割、旋转、数据类型转换、图像到tensor、numpy数组到tensor、tensor到图像等
torchvision.utils	用于将形似 $(3 \times H \times W)$ 的张量保存到硬盘中，能够制作图像网络

其中torchvision.transforms中常用的5种图像预处理操作如下。

1. 组合图像的多种变换处理

- Compose类可以将多种图像的变换处理组合到一起，Compose类语法格式如下。
- 其中参数“transforms”接收的是由多种变换处理组合成的列表。

torchvision.transforms.Compose(transforms)

2. 组合图像的多种变换处理

```
from torchvision import transforms
```

```
transforms.Compose([transforms.CenterCrop(10),  
                    transforms.ToTensor()])
```

2. 对图像做变换处理

常见的PIL图像包括以下几种模式。

- L（灰色图像）
- P（8位彩色图像）
- I（32位整型灰色图像）
- F（32位浮点灰色图像）
- RGB（8位彩色图像）
- YCbCr（24位彩色图像）
- RGBA（32位彩色模式）
- CMYK（32位彩色图像）
- 1（二值图像）

图像数据预处理

在PyTorch框架下能实现对PIL图像和torch张量做变换处理的类较多，此处仅介绍常用的6个类。

➤ CenterCrop类

- CenterCrop类可以裁剪给定的图像并返回图像的中心部分。
- 如果图像是torch张量，形状将会是[$\dots$, H, W]，其中省略号“ $\dots$ ”是一个任意尺寸。
- 如果输入图像的大小小于期望输出图像的大小，则在输入图像的四周填充0，然后居中裁剪。
- CenterCrop类的语法格式如下，其中参数“size”指的是图像的期望输出大小。

torchvision.transforms.CenterCrop(size)

➤ ColorJitter类

- ColorJitter类可以随机改变图像的亮度、对比度、饱和度和色调。如果图像是torch张量，形状将会是[$\dots$, 3, H, W]，其中省略号“ $\dots$ ”表示任意数量的前导维数。
- 如果图像是PIL图像，则不支持模式1、L、I、F和透明模式。
- ColorJitter的语法格式如下。

torchvision.transforms.ColorJitter(brightness=0, contrast=0, saturation=0, hue=0)

图像数据预处理

ColorJitter类的常用参数及其说明如表所示。

参数名称	说明
brightness	接收int，表示亮度大小。亮度因子统一从【最大值（0， 1-自定义值）， 1+亮度】或给定的【最小值，最大值】中选择。默认为0
contrast	接收int，表示对比度大小。对比度因子统一从【最大值（0， 1-自定义值）， 1+对比度】或给定【最小值，最大值】中选择。默认为0
saturation	接收int，表示饱和度大小。饱和度因子统一从【最大值（0， 1-自定义值）， 1+饱和度】或给定【最小值，最大值】中选择。应该是非负数。默认为0
hue	接收int，表示色调大小。色调因子统一从【-自定义值，自定义值】或给定的【最小值，最大值】中选择。且有 $0 \leq \text{自定义值} < 0.5$ 或 $-0.5 \leq \text{最小值} \leq \text{最大值} \leq 0.5$ 。默认为0

➤ FiveCrop类

- FiveCrop类可以将图像裁剪成四个角和中心部分。
- 如果图像是torch张量，形状将会是[$\dots$, H, W]，其中“ $\dots$ ”表示任意数量的前导维数。
- FiveCrop类的语法格式如下，其中参数“size”指的是裁剪图像的期望大小。

torchvision.transforms.FiveCrop(size)

➤ Grayscale类

- Grayscale类可以将图像转换为灰度图像。
- 如果图像是torch张量，形状将会是[$\dots$, 3, H, W]，其中省略号“ $\dots$ ”表示任意数量的前导维数。
- Grayscale类的语法格式如下，其中参数“num_output_channels”表示的是输出图像所需的通道数。

`torchvision.transforms.Grayscale(num_output_channels=1)`

➤ Pad类

- Pad类可以使用给定的填充值填充图像的边缘区域。
- 如果图像是torch张量，形状将会是[...， H， W]，其中省略号“...”表示模式反射和对称的最多2个前导维数。
- Pad类的语法格式如下。

`torchvision.transforms.Pad(padding, fill=0, padding_mode='constant')`

图像数据预处理

➤ Pad类的常用参数及其说明如表所示。

参数名称	说明
padding	接收int或序列，表示在图像边缘区域填充。如果只提供一个整型int数据，那么填充所有的边缘区域。如果提供了长度为2的序列，那么填充左右边缘或上下边缘区域。如果提供了长度为4的序列，那么填充上下左右四个边缘区域。无默认值
fill	接收int或元组，表示填充值。如果值是长度为3的元组，那么分别用于填充R、G、B三个通道。此值仅在填充模式为常数时使用。默认为0。

➤ Resize类

- Resize类可以将输入图像调整到指定的尺寸。
- 如果图像是torch张量，形状将会是[..., H, W]形状，其中省略号“...”表示任意数量的前导维数。
- Resize类的语法格式如下。

torchvision.transforms.Resize(size,
interpolation=<InterpolationMode.BILINEAR: 'bilinear'>)

图像数据预处理

- Resize类的常用参数及其说明如表所示。

参数名称	说明
size	接收int，表示期望输出大小，无默认值
interpolation	接收str，表示插入式模式。默认为InterpolationMode.BILINEAR

3. 对变换处理列表做处理

PyTorch不仅可设置对图片的变换处理，还可以对这些变换处理进行随机选择、组合，数据增强更灵活。

➤ RandomChoice类

- RandomChoice类从接收的变换处理列表中随机选取的单个变换处理对图像进行变换。
- RandomChoice类的语法格式如下。
- RandomChoice类的参数说明与Compose类一致。

torchvision.transforms.RandomChoice(transforms)

图像数据预处理

➤ RandomOrder类

- RandomOrder类随机打乱接收的变换处理列表中的变换处理。
- RandomOrder类的语法格式如下。
- RandomOrder类的参数说明与Compose类一致。

torchvision.transforms.RandomOrder(transforms)

4. 对图像数据做变换处理

对图像数据做变换处理操作的类较多，此处介绍常用的两个类。

➤ LinearTransformation类

- LinearTransformation类用平方变换矩阵和离线计算的均值向量变换图像数据。
- LinearTransformation类的语法格式如下。

*torchvision.transforms.LinearTransformation(transformation_matrix,
mean_vector)*

4. 对图像数据做变换处理

➤ LinearTransformation类的常用参数及其说明如表所示。

参数名称	说明
transformation_matrix	接收str，表示变换矩阵，输入格式为[DxD]。无默认值
mean_vector	接收str，平均向量，输入格式为[D]。无默认值

➤ Normalize类

- Normalize类用均值和标准差对图像数据进行归一化处理。
- Normalize类的语法格式如下。

`torchvision.transforms.Normalize(mean, std, inplace=False)`

➤ Normalize类的常用参数及其说明如表所示。

参数名称	说明
mean	接收int或float，表示每个通道的均值。无默认值
std	接收int或float，表示每个通道的标准差。无默认值

5. 格式转换变换处理

由于直接读取图像得到的数据的类型与输入网络的数据类型不对应，因此需要对数据的格式进行转换。

➤ ToPILImage类

- ToPILImage类将tensor类型或ndarray类型的数据转换为PIL Image类型的数据。
- ToPILImage类的语法格式如下，其中参数“mode”指的是输入数据的颜色空间和像素深度。

torchvision.transforms.ToPILImage(mode=None)

➤ ToTensor类

- ToTensor类将PIL Image类型或ndarray类型的数据转换为tensor类型的数据，并且归一化至[0,1]。
- ToTensor类的语法格式如下。

torchvision.transforms.ToTensor

- 如果PIL图像属于L、P、I、F、RGB、YCbCr、RGBA、CMYK和1这些形式之一，那么将[0,255]范围内的PIL图像或numpy.ndarray($H \times W \times C$)转换为[0.0,1.0]范围内的浮点张量（ $C \times H \times W$ ）。
- 在其他情况下，不按比例返回张量。

在PyTorch框架中可以利用torchtext库中的相关类对文本数据进行数据预处理。

1. 划分训练集和测试集

- SogouNews类可以对PyTorch框架自带的数据集进行训练集测试集的划分，SogouNews类的语法格式如下。

torchtext.datasets.SogouNews(root='.data', split=('train', 'test'))

- SogouNews类的常用参数及其说明如表所示。

参数名称	说明
root	接收str，表示保存数据集的目录，无默认值
split	接收str，表示划分数据集和测试集的标准，无默认值

- WikiText2类也可以对PyTorch框架自带的数据集进行训练集、验证集和测试集的划分。

- WikiText2类的语法格式如下。

torchtext.datasets.WikiText2(root='.data', split=('train', 'valid', 'test'))

- WikiText2类仅能对三个PyTorch自带数据集进行操作，分别是WikiText-2数据集、WikiText103数据集和PennTreebank数据集。

2. 创建词汇表

- Vocab类可以定义用于计算字段的词汇表对象。
- Vocab类的语法格式如下。

```
torchtext.vocab.Vocab(counter, max_size=None, min_freq=1,  
pecials=('<unk>', '<pad>'), vectors=None, unk_init=None,  
vectors_cache=None, specials_first=True)
```

Vocab类的常用参数及其说明如表所示。

参数名称	说明
counter	接收float，表示保存数据中每个值的频率，无默认值
max_size	接收int，表示最大词汇容量，无默认值
min_freq	接收int，表示能保存进词汇表的最低频率，默认为1
specials	接收str，表示将加在词汇表前面的特殊标记的列表，默认值为('<unk>','<pad>')
vectors	接收int，表示可用的预训练向量，无默认值
vectors_cache	接收str，表示缓存向量的目录，无默认值

3. 构建句子生成器

- generate_sp_model类用于构建句子生成器， generate_sp_model类的语法格式如下。

```
torchtext.data.functional.generate_sp_model(filename,  
vocab_size=20000, model_type='unigram', model_prefix='m_user')
```

3. 构建句子生成器

➤ generate_sp_model类的常用参数及其说明如表所示。

参数名称	说明
filename	接收str，表示用于构建句子生成器的数据文件，无默认值
vocab_size	接收int，表示词汇量，默认为20000
model_type	接收str，表示生成句子网络的类型，包括unigram、bpe、char、word，默认为unigram
model_prefix	接收str，表示数据文件和词汇表的前缀保存形式，默认为m_user

4. 加载句子生成器

- load_sp_model类用于加载数据文件的句子生成器。
- load_sp_model类的语法格式如下，其中参数“SPM”指的是保存句子生成器的文件路径。

torchtext.data.functional.load_sp_model(SPM)

5. 构建句子计数器

- `sentencepiece_numericalizer`类用于建立基于文本句子的数字映射。
- `sentencepiece_numericalizer`类的语法格式如下，其中参数“`sp_model`”指的是句子生成器。

`torchtext.data.functional.sentencepiece_numericalizer(sp_model)`

6. 构建句子标记器

- `sentencepiece_tokenizer`类用于标记文本句子中各个词汇，`sentencepiece_tokenizer`类的语法格式如下。
- 输入文本句子，将输出句子中所有成分。

`torchtext.data.functional.sentencepiece_tokenizer(sp_model)`

7. 构建句子分词器

- `simple_space_split`类用于对句子进行分词处理，并输出句子中的各个词汇。
- `simple_space_split`类的语法格式如下。

`torchtext.data.functional.simple_space_split(iterator)`

目录



加载及预处理猫狗分类数据

- 本小节将以猫狗数据集为例进行演示数据加载和预处理。
- 猫狗数据集的训练集包含25000张图片，其中猫图有12500张，狗图有12500张。
- 测试集包含400张图片，其中猫图有200张，狗图有200张。
- 定义路径读取函数，用于得到一个包含所有图片文件名（包含路径）和标签（狗1猫0）的列表。

加载及预处理猫狗分类数据

```
import torch
from torchvision import transforms
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import torch.nn.functional as F

def init_process(path, lens):
    data = []
    name = find_label(path)
    for i in range(lens[0], lens[1]):
        data.append([path % i, name])

    return data
```

加载及预处理猫狗分类数据

- 现有数据的命名都是有序号的，训练集中数据编号为0~499，测试集中数据编号为1000~1200，因此可以根据这个规律获取训练集图像路径的列表。

```
path1 = '../data/training_data/cats/cat.%d.jpg'  
data1 = init_process(path1, [0, 500])
```

- data1对象是一个包含五百个文件名以及标签的列表。find_label函数用于判断标签是dog（狗）还是cat（猫），dog返回1，cat返回0。

加载及预处理猫狗分类数据

```
def find_label(str):
    first, last = 0, 0
    for i in range(len(str) - 1, -1, -1):
        if str[i] == '%' and str[i - 1] == '.':
            last = i - 1
        if (str[i] == 'c' or str[i] == 'd') and str[i - 1] == '/':
            first = i
            break

    name = str[first: last]
    if name == 'dog':
        return 1
    else:
        return 0
```

加载及预处理猫狗分类数据

- 在定义了路径读取和标签判读函数后，调用四次路径读取和标签判读函数并输入不同的参数，即可获取到四个列表（训练集中的猫、训练集中的狗、测试集中的猫、测试集中的狗）。

```
path1 = '../data/training_data/cats/cat.%d.jpg'
data1 = init_process(path1, [0, 500])
path2 = '../data/training_data/dogs/dog.%d.jpg'
data2 = init_process(path2, [0, 500])
path3 = '../data/testing_data/cats/cat.%d.jpg'
data3 = init_process(path3, [1000, 1200])
path4 = '../data/testing_data/dogs/dog.%d.jpg'
data4 = init_process(path4, [1000, 1200])
```

加载及预处理猫狗分类数据

- 使用PIL包的Image类读取图片。

```
def Myloader(path):  
    return Image.open(path).convert('RGB')
```

加载及预处理猫狗分类数据

- 重写PyTorch的Dataset类，其中__getitem__()方法用于读取数据集中数据，并对数据进行变换操作。

```
class MyDataset(Dataset):  
    def __init__(self, data, transform, loader):  
        self.data = data  
        self.transform = transform  
        self.loader = loader  
    def __getitem__(self, item):  
        img, label = self.data[item]  
        img = self.loader(img)  
        img = self.transform(img)  
        return img, label  
    def __len__(self):  
        return len(self.data)
```

加载及预处理猫狗分类数据

- 使用transform对象中包含的变换处理操作如下。
 - `transforms.RandomHorizontalFlip (p=0.3)`, 每张图片有30%的概率水平翻转。
 - `transforms. RandomVerticalFlip (p=0.3)`, 每张图片有30%的概率垂直翻转。
 - `transforms.ToTensor()`, 很重要的一步, 将图像数据转为Tensor类型。
 - `transforms.Normalize(mean=(0.5,0.5,0.5), std=(0.5,0.5,0.5))`, 归一化。

```
transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.3),
    transforms.RandomVerticalFlip(p=0.3),
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5))])
```

课后作业

- 整理猫狗分类数据加载和预处理代码，写为一个代码
- 返回train_data以及test_data，即为最终需要得到的数据。